Relatório de Cobertura de Testes — REA (Reverse Engineer Anything)

Aluno: Italo Dell Areti
Repositório analisado: github.com/morluto/rea (≈211 mil estrelas, ≈51,8 mil forks)
Commit analisado: `41a2e84` (14/10/2026)
Linguagem: TypeScript (Node.js)
Ferramenta de cobertura: Vitest 5.0.3 + `@vitest/coverage-v8` (cobertura nativa do V8, relatório HTML no formato Istanbul)

1. O projeto

O REA é um servidor MCP (*Model Context Protocol*) e uma CLI que permitem a agentes de IA fazer engenharia reversa de binários nativos, aplicativos (Android, Apple, .NET, Flutter, WebAssembly etc.), navegadores e processos em execução. Para isso, integra ferramentas externas como Ghidra, Hopper, IDA, rizin, LLDB, GDB, Frida e Playwright. O código-fonte (`src/`) tem 1.386 arquivos TypeScript instrumentados e 73.437 linhas executáveis.

A arquitetura é em camadas (`domain`, `contracts`, `application`, adaptadores por ferramenta e `server`/`cli`), e a suíte de testes segue essa divisão. São 12 *projects* do Vitest (`domain`, `services`, `adapters`, `composition`, `boundary`, `cli-boundary`, `process-boundary`, `mcp-boundary`, `acceptance`, `process-global`, `conformance` e `evaluation`) com 1.186 arquivos de teste.

2. Execução dos testes

Ambiente: Linux x86-64, Node.js 24.21.0 (o projeto exige `^24.11` ou `>=26`), npm 11.

```
npm ci
npm run test:coverage -- --coverage.reportOnFailure=true \
  --coverage.reporter=html --coverage.reporter=json-summary \
  --coverage.reporter=text-summary --coverage.reportsDirectory=out/rea/coverage
```

`test:coverage` é o script oficial do projeto (`vitest run --coverage.enabled --coverage.provider=v8`). O script `scripts/rea-coverage.sh`, neste repositório, reproduz a execução completa. Foi preciso adicionar `--coverage.reportOnFailure=true`, porque por padrão o Vitest **não gera o relatório** quando algum teste falha. Isso aconteceu na primeira execução.

Resultado	Arquivos de teste	Casos de teste
Aprovados	1.170	12.832
Falharam	3	5
Ignorados (*skipped*)	13	118
Total	**1.186**	**12.955**

Duração: cerca de 48 minutos, com no máximo 2 *workers*, limite definido pelo próprio `vitest.config.ts`.

Falhas. As 5 falhas são de ambiente, não do código:

- **2 em `process-boundary`** (`processFilesystemCoverageAdapters.test.ts`): o REA verifica, via `/proc/<pid>/environ`, se os processos filhos pertencem à captura. A execução ocorreu em um *sandbox* que nega essa leitura (`EACCES`), então a limpeza termina como `owned_process_group=unverified` e o teste falha.
- **3 em `mcp-boundary`** (`serverIdentity.test.ts`, `sessionMcp.test.ts`): estouraram o *timeout* de 10 s por causa da lentidão introduzida pela instrumentação de cobertura.

Elas são instáveis (*flaky*). Uma execução anterior, idêntica, falhou em **outros** 2 testes de `process-boundary` com o mesmo erro `EACCES` e passou nos 3 testes MCP.

Testes ignorados. Os 13 arquivos ignorados usam `skipIf(process.platform !== "darwin")`: dependem de macOS (plist, Mach-O, assinatura de código, NIB, inicialização do Hopper) e não rodam em Linux.

3. Resultados gerais

Métrica	Cobertos / Total	Cobertura	Limite do projeto
Statements	70.126 / 79.233	**88,50%**	65%
Branches	50.703 / 62.653	**80,92%**	60%
Functions	12.360 / 13.504	**91,52%**	60%
Lines	66.097 / 73.437	**90,00%**	68%

O próprio `vitest.config.ts` define limites mínimos (*thresholds*), e todos foram superados com folga, inclusive os mais rígidos para as camadas centrais:

Diretório	Statements	Branches	Functions	Lines	Limite (L/B)
`src/domain/**`	92,9%	86,5%	97,3%	94,3%	80% / 75%
`src/contracts/**`	95,8%	87,6%	97,7%	97,6%	85% / 80%

Distribuição por arquivo (linhas): 558 arquivos (40%) têm 100% de cobertura de linhas, 374 deles também com 100% de *branches*; 453 ficam entre 90% e 100%; 229 entre 70% e 90%; 37 entre 50% e 70%; e 109 abaixo de 50%.

4. Cobertura por módulo

Os módulos abaixo são diretórios de primeiro nível de `src/` com pelo menos 400 linhas:

Módulo	Arquivos	Linhas cobertas	Linhas	Branches
`rizin`	2	528 / 532	99,2%	97,8%
`contracts`	76	830 / 850	97,6%	87,6%
`gdb`	1	643 / 669	96,1%	91,9%
`ida`	13	787 / 821	95,9%	90,2%
`flutter`	6	947 / 1.000	94,7%	86,2%
`domain`	397	20.168 / 21.376	94,3%	86,5%
`server` (MCP)	89	1.814 / 1.957	92,7%	77,1%
`dotnet`	26	2.106 / 2.273	92,7%	82,2%
`artifacts`	46	3.709 / 4.036	91,9%	80,5%
`ghidra`	45	2.400 / 2.630	91,3%	84,5%
`process`	43	2.560 / 2.821	90,7%	79,8%
`application`	244	10.816 / 11.939	90,6%	82,6%
`native`	45	2.938 / 3.363	87,4%	74,9%
`browser`	92	5.301 / 6.214	85,3%	75,8%
`javascript`	20	1.128 / 1.330	84,8%	73,6%
`android`	9	582 / 726	80,2%	66,7%
`hopper`	22	977 / 1.414	**69,1%**	62,0%
`cli`	50	742 / 1.394	**53,2%**	**30,8%**

O núcleo (`domain` e `contracts`), onde está a lógica de negócio pura, é o mais bem testado e concentra 30% de todas as linhas do projeto. Os módulos com menor cobertura ficam na periferia: a CLI interativa e o adaptador do Hopper.

5. Arquivos com mais e menos cobertura

Mais cobertos. Vários arquivos grandes têm 100% de linhas e de *branches*, como `application/DoctorDiagnostics.ts` (80 linhas), `process/ExecFileOutput.ts` (68), `ghidra/GhidraInventoryValues.ts` (56), `domain/analysisErrorCore.ts` (49), `domain/canonicalDigest.ts` (49) e `server/registerBrowserTools.ts` (45).

Mais linhas não cobertas:

Arquivo	Linhas não cobertas	Cobertura
`domain/native/nativeAotPe.ts`	176	75,6%
`hopper/HopperFileOffset.ts`	126	3,1%
`cli/interactiveSetup.ts`	112	16,4%
`application/ClientConfigurationDocument.ts`	92	87,0%
`hopper/HopperMachOImage.ts`	91	13,3%
`native/LldbCallTracer.ts`	85	61,9%
`adb/AdbProvider.ts`	76	85,1%
`android/JadxWorkspaceRoot.ts`	71	9,0%
`application/GeminiClientSettings.ts`	65	12,2%
`browser/history/SazCapture.ts`	65	0,0%

6. O que não é coberto, e por quê

Na análise do relatório HTML, o código não coberto se encaixa em quatro categorias:

a) Código que roda em *worker threads* ou processos filhos (falso 0%). Dos 12 arquivos com 0%, 11 são pontos de entrada de *workers*: `HopperRegexWorker.ts`, `SqliteDatabaseWorker.ts`, `EvmoleInterfaceWorker.ts`, `JavaScriptAnalysisWorkerServer.ts`, `SourceMapCodecProcess.ts`, `NodeCaptureProcess.ts` e o pacote `browser/history/Saz*`, que só é carregado por `NodeCaptureProcess.ts`. O 12º é `toolContractTypes.ts`, que só tem tipos. Os testes exercitam esses módulos, mas eles rodam em outro isolado V8 (`node:worker_threads`) ou em outro processo, e o *provider* V8 só coleta a cobertura do processo principal do *worker* de teste. Assim, a cobertura real é maior do que a medida.

b) Integrações com ferramentas externas e com plataformas específicas. `hopper/` (69%) depende do Hopper Disassembler, um produto comercial. `HopperFileOffset.ts` (3%) e `HopperMachOImage.ts` (13%) só rodam com o Hopper instalado, e `JadxWorkspaceRoot.ts` (9%) só com o JADX. O projeto valida esses caminhos com scripts separados (`npm run verify:hopper`, `verify:android`, `verify:ghidra`...) contra as ferramentas reais, fora do Vitest e, portanto, fora deste relatório. Os testes exclusivos de macOS, ignorados em Linux, também deixam sem cobertura parte de `native/` e `artifacts/apple/` (por exemplo, `NibArchive.ts`, com 74,6%).

c) Interface interativa de terminal. `cli/` tem a pior cobertura de *branches* (30,8%). `interactiveSetup.ts` monta o assistente `rea setup` com prompts (`intro`, `confirm`, seleção de clientes), que exigem um TTY real. Os testes de CLI usam o modo não interativo, então todo o fluxo de seleção e confirmação (linhas 40–56, 97–103 e 245–304) fica sem cobertura. O mesmo vale para a leitura e escrita de configurações de clientes específicos, como `GeminiClientSettings.ts` (12%).

d) Tratamento de erros e caminhos defensivos. É a explicação para a diferença de cerca de 9 pontos entre linhas (90%) e *branches* (81%). Em `nativeAotPe.ts`, o parser de binários .NET NativeAOT, quase todas as linhas não cobertas são verificações defensivas que só disparam com binários malformados ou maliciosos. Exemplo das linhas 845–863:

```
if (!budget.consumeWork(4))
  throw new Error("NativeAOT parser work budget is exhausted");
if (delta < -0x80000000n || delta > 0x7fffffffn)
  throw new Error("Hydrated relative pointer does not fit int32");
...
if (target === null)
  throw new Error("DEHYDRATED_DATA fixup leaves captured PE sections");
```

Em `LldbCallTracer.ts` (62%), o bloco `catch` que trata falhas ao ler e fechar o arquivo de captura do LLDB (linhas 769–783) também não é exercitado.

7. Conclusão

O REA tem **cobertura alta**: 90% das linhas e 81% dos *branches*, acima dos limites que o próprio projeto exige no CI. A estratégia de testes é coerente com a arquitetura, com o núcleo (`domain`, `contracts`) acima de 94% de linhas. Os pontos fracos estão na periferia, onde o teste automatizado é mais caro: a CLI interativa, os adaptadores de ferramentas comerciais (Hopper) e o código exclusivo de macOS. Parte dos 0% é artefato da ferramenta (os *workers*), não falta de teste.

Para melhorar, as recomendações são:

1. Tornar injetáveis os *streams* de entrada e saída de `interactiveSetup.ts` (hoje fixos na constante `promptStreams`) para testar os prompts com *streams* simulados.
2. Escrever testes de propriedade ou *fuzzing* com binários PE malformados para `nativeAotPe.ts`. O projeto já usa `@fast-check/vitest`.
3. Incluir a cobertura dos *workers* (por exemplo, com `NODE_V8_COVERAGE` nos processos filhos) para que o relatório reflita a cobertura real.

Anexo

O relatório HTML gerado pela ferramenta está em `rea-cobertura-html.zip`. Para navegar, abra `rea-coverage-html/index.html`.